



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Сервер для отдачи статического содержимого»

Студент ИУ7-74Б
(Группа)

(Подпись, дата)

Сальников М. А.
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Клочков М. Н.
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ОПРЕДЕЛЕНИЯ	6
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
ВВЕДЕНИЕ	8
1 Аналитический раздел	9
1.1 Анализ предметной области	9
1.2 Сетевой протокол	9
1.2.1 Протокол HTTP	9
1.3 Классы моделей ввода-вывода	10
1.4 Механизм prefork	11
1.5 Механизм select	12
2 Конструкторский раздел	13
2.1 Требования к серверу	13
2.2 Архитектура сервера	13
2.2.1 Диаграмма контекста	13
2.2.2 Диаграмма контейнеров	14
2.2.3 Диаграммы компонентов	15
2.3 Алгоритмы работы сервера	17
2.3.1 Алгоритм запуска сервера	17
2.3.2 Алгоритм передачи нового соединения от главного про- цесса к рабочему	18
2.3.3 Алгоритм обработки запроса к серверу	19
2.4 Выбор типов и структур данных	20
2.5 Классы эквивалентности для тестирования	22
3 Технологический раздел	23
3.1 Средства реализации	23
3.2 Демонстрация работы сервера	23
3.3 Тестирование	27

4	Исследовательский раздел	29
4.1	Постановка исследования	29
4.2	Технические характеристики	29
4.3	Проведение исследования	29
4.3.1	Максимальное количество обслуживаемых сетевых со- единений	29
4.3.2	Скорость отдачи данных по сетевому соединению	31
4.3.3	Совокупная скорость отдачи данных по всем сетевым соединениям	33
	ЗАКЛЮЧЕНИЕ	35
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
	ПРИЛОЖЕНИЕ А	37

РЕФЕРАТ

Расчетно-пояснительная записка 36 с., 9 рис., 4 лист., 9 ист., 1 прил.

КОМПЬЮТЕРНЫЕ СЕТИ, СТАТИЧЕСКИЙ СЕРВЕР, C4, ПРОТОКОЛ HTTP, PREFORK, SELECT, LINUX, C.

Цель работы: разработка сервера для отдачи статического содержимого с диска по протоколу HTTP с использованием механизмов prefork и select.

В данной работе разрабатывается сервер для отдачи статического содержимого. Проводится анализ предметной области и анализ используемого набора механизмов. Проектируется архитектура в нотации моделирования C4. Проводится реализация на языке программирования C, тестирование сервера и исследуется его работа с использованием программы Apache Benchmark.

В результате работы разработан сервер для отдачи статического содержимого с диска по протоколу HTTP с использованием механизмов prefork и select. Работоспособность системы протестирована, исследована и подтверждена на реальных сценариях.

ОПРЕДЕЛЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие термины с соответствующими определениями.

Логгер — специальный модуль, библиотека или программа, записывающая события происходящие на сервере в журнал событий.

Path-traversal атака — попытка получить доступ к файлам и директориям вне корневой папки статического сервера.

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие сокращения и обозначения.

MIME — Multipurpose Internet Mail Extensions

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка сервера для отдачи статического содержимого с диска по протоколу HTTP с использованием механизмов `prefork` и `select`.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- провести анализ предметной области;
- провести анализ используемого набора механизмов;
- спроектировать архитектуру разрабатываемого сервера;
- выбрать средства реализации и реализовать сервер;
- провести тестирование разработанного сервера;
- провести исследование разработанного сервера.

1 Аналитический раздел

В данном разделе представляются анализ предметной области, понятия сетевого протокола, классов моделей ввода-вывода, видов запросов к серверу и описания механизмов `prefork` и `select`.

1.1 Анализ предметной области

Статический сервер – это программное обеспечение, предназначенное для предоставления пользователям через сеть статического содержимого по их запросу. Статическое содержимое представляет из себя неизменяемые данные, такие как HTML-страницы, изображения, файлы.

1.2 Сетевой протокол

Для обеспечения передачи данных по организованным каналам связи в инфокоммуникационных системах применяются протоколы. Протокол – набор соглашений интерфейса логического уровня, которые определяют обмен данными между различными программами. По сути задача протокола – предоставить передаваемые пользователем данные в приемлемой форме и сопроводить служебной информацией, которая позволит корректно передать по каналам связи и интерпретировать данные на принимаемой стороне [1].

Сетевой протокол – набор правил и действий (очередности действий), позволяющий осуществлять соединение и обмен данными между двумя и более включёнными в сеть устройствами.

1.2.1 Протокол HTTP

HTTP представляет собой сетевой протокол прикладного уровня, изначально разработанный для обеспечения передачи гипертекстовых документов формата HTML с удалённых серверов. В процессе эволюции протокол трансформировался в универсальный механизм взаимодействия между узлами распределённых информационных систем, охватывающих как глобальную инфраструктуру Всемирной паутины, так и локальные веб-инфраструктуры [2].

Функциональное назначение протокола заключается в обеспечении передачи информационного содержимого в глобальной сети Интернет посредством стандартизированного формата взаимодействия между клиентом и сервером.

Структурная организация протокола предусматривает стандартизированный формат как для запросов, так и для ответов. Запрос формируется из строки запроса, набора заголовков, разделительной пустой строки и тела запроса. Ответ состоит из строки статуса, заголовков ответа, пустой строки и тела ответа.

Некоторыми из существующих HTTP-методов являются:

1. **GET** – метод для запроса содержимого указанного ресурса. Гарантирует идемпотентность операции и используется для получения данных без изменения состояния сервера;
2. **HEAD** – метод для запроса только заголовка ответа без тела сообщения. Применяется для получения метаданных о ресурсе без загрузки его содержимого.

Статусы данного протокола:

1. **1XX** – информационный;
2. **2XX** – успешное выполнение;
3. **3XX** – перенаправление;
4. **4XX** – ошибка на стороне клиента;
5. **5XX** – ошибка на стороне сервера.

1.3 Классы моделей ввода-вывода

Модель ввода-вывода определяет механизм взаимодействия процессов с устройствами, порядок обработки запросов, способ блокировки операций, управление ресурсами и буферизацией. Выделяют четыре класса моделей ввода-вывода [3]:

1. *Синхронный блокирующий ввод-вывод* характеризуется полной блокировкой процесса до момента завершения операции ввода-вывода. При выполнении запроса процесс приостанавливается до получения данных или кода ошибки. Все операции реализуются через системные вызовы, которые переводят систему в режим ядра. Ключевая особенность данного класса заключается в том, что процесс остаётся заблокированным до полного завершения операции;

2. *Синхронный неблокирующий ввод-вывод* основан на механизме периодического опроса состояния устройства. В отличие от блокирующего аналога, процесс не приостанавливается, однако тратит процессорное время на постоянный мониторинг готовности данных. При этом система немедленно возвращает результат попытки выполнения операции, не дожидаясь фактической готовности данных;
3. *Асинхронный блокирующий ввод-вывод* (мультиплексирование) представляет собой способ обработки множества операций ввода-вывода в рамках одного процесса. Модели данного класса используют специальные механизмы для одновременного отслеживания состояния нескольких источников данных, что позволяет процессу блокироваться только при ожидании готовности любого из отслеживаемых ресурсов;
4. *Асинхронный неблокирующий ввод-вывод* реализуется через два подхода. Первый основан на использовании сигналов, которые уведомляют процесс о готовности данных без его блокировки. Вторым подход предполагается запуск операции ввода-вывода с последующим получением уведомления о её завершении. В обоих случаях процесс сохраняет возможность выполнения других задач, получая уведомления о завершении операций по мере их готовности.

1.4 Механизм prefork

Механизм prefork представляет из себя шаблон проектирования многопоточных серверов, основанный на предварительном создании дочерних процессов до поступления клиентских запросов.

Принцип работы заключается в том, что родительский процесс создаёт заранее определённое количество дочерних процессов, которые находятся в состоянии ожидания входящих соединений. При поступлении запроса один из свободных дочерних процессов обрабатывает его, после чего возвращается в состояние ожидания следующего запроса.

Преимущества данного механизма:

- изолированность обработки запроса клиента;
- отсутствие необходимости использования средств взаимного исключения.

Недостатки данного механизма:

- необходимость выделения памяти под каждый из процессов;
- затраты времени на создание и инициализацию каждого из процессов.

1.5 Механизм select

Механизм select относится к классу асинхронных блокирующих моделей ввода-вывода. Он позволяет процессу одновременно отслеживать состояние множества файловых дескрипторов и определять их готовность к операциям чтения, записи или наличию исключительных ситуаций.

Принцип работы основан на передаче ядру набора файловых дескрипторов и ожидании изменения их состояния. Процесс блокируется до тех пор, пока хотя бы один из дескрипторов не перейдёт в интересующее состояние или не истечёт заданный промежуток времени.

Преимущества данного механизма:

- возможность одновременного отслеживания множества соединений;
- процесс блокируется только при отсутствии готовых к обработке дескрипторов.

Недостатки данного механизма:

- процесс блокируется на время ожидания готовности дескрипторов;
- ограничение на максимальное количество отслеживаемых дескрипторов.

Вывод

В данном разделе были представлены анализ предметной области, понятия сетевого протокола, классов моделей ввода-вывода, видов запросов к серверу и описания механизмов `prefork` и `select`.

2 Конструкторский раздел

В данном разделе представляются требования к серверу, архитектура сервера, алгоритмы работы сервера, выбор типов и структур данных и классы эквивалентности для тестирования.

2.1 Требования к серверу

Разрабатываемый сервер должен отвечать следующим требованиям:

1. поддержка запросов GET и HEAD;
2. поддержка статусов, соответствующих успешному выполнению и ошибкам на стороне клиента;
3. поддержка передачи файлов до 128 Мбайт;
4. использование механизма мультиплексирования;
5. использование логирования.

2.2 Архитектура сервера

Архитектура сервера представлена в нотации моделирования архитектуры программных систем С4 [4].

2.2.1 Диаграмма контекста

Диаграмма контекста демонстрирует, как программная система взаимодействует с внешними сущностями.

Диаграмма контекста разрабатываемого сервера представлена на рисунке 2.1.

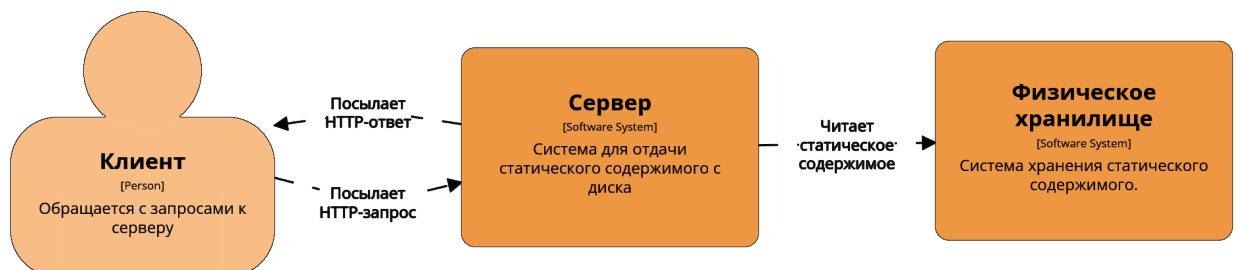


Рисунок 2.1 – Диаграмма контекста

2.2.2 Диаграмма контейнеров

Диаграмма контейнеров описывает верхнеуровневую архитектуру и технологии. Используется для понимания технологического стека и разделения зон ответственности.

Диаграмма контейнеров разрабатываемого сервера представлена на рисунке 2.2.

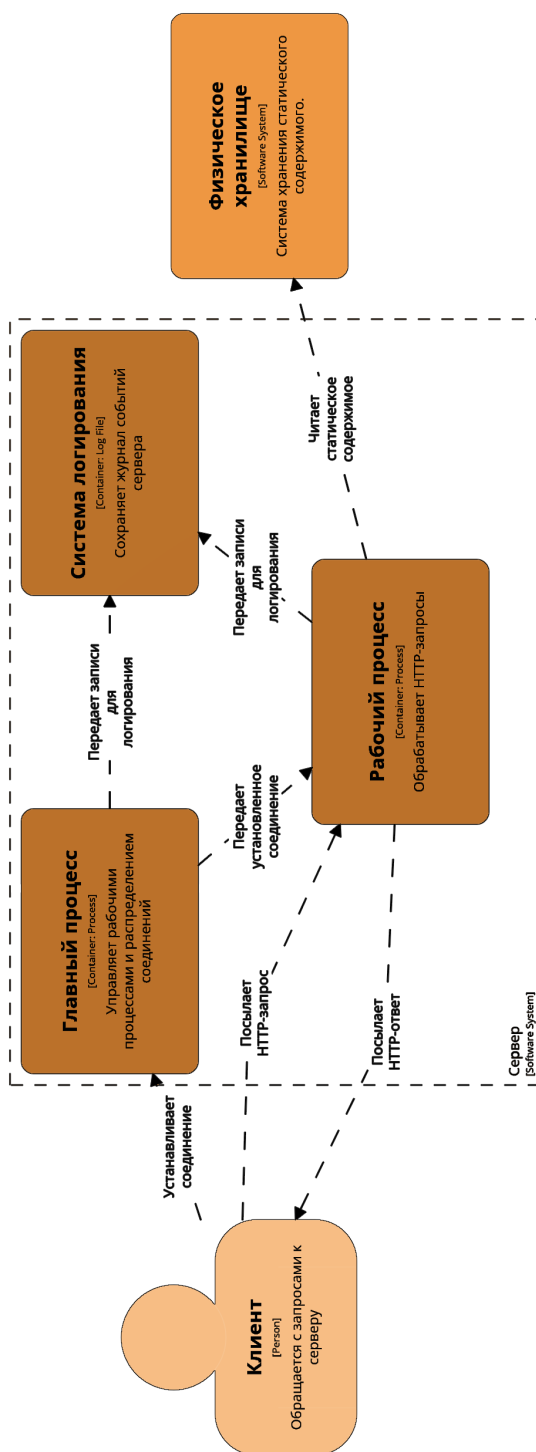


Рисунок 2.2 – Диаграмма контейнеров

2.2.3 Диаграммы компонентов

Диаграмма компонентов детализирует структуру внутри контейнера системы, то есть более детально описывает один из контейнеров из предыдущего уровня.

Диаграмма компонентов главного процесса представлена на рисунке 2.3.

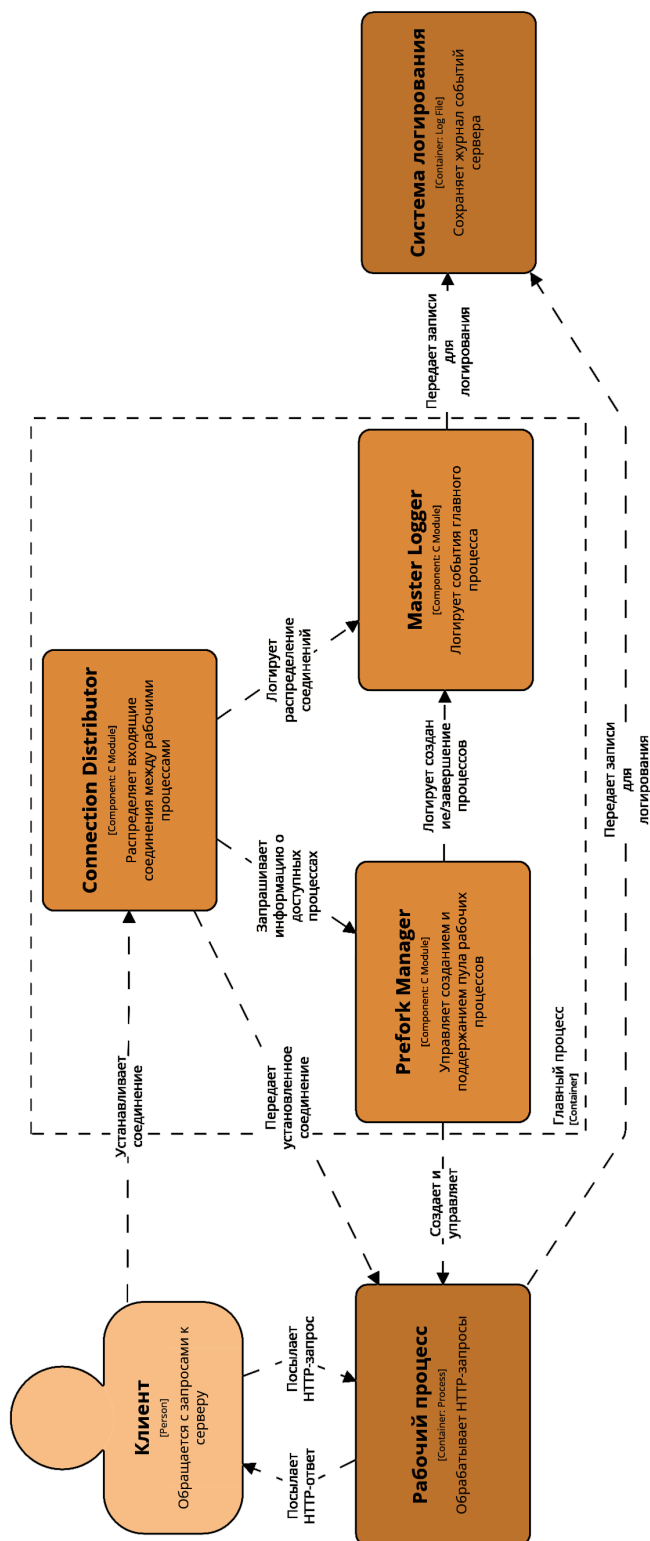


Рисунок 2.3 – Диаграмма компонентов главного процесса

Диаграмма компонентов рабочего процесса представлена на рисунке 2.4.

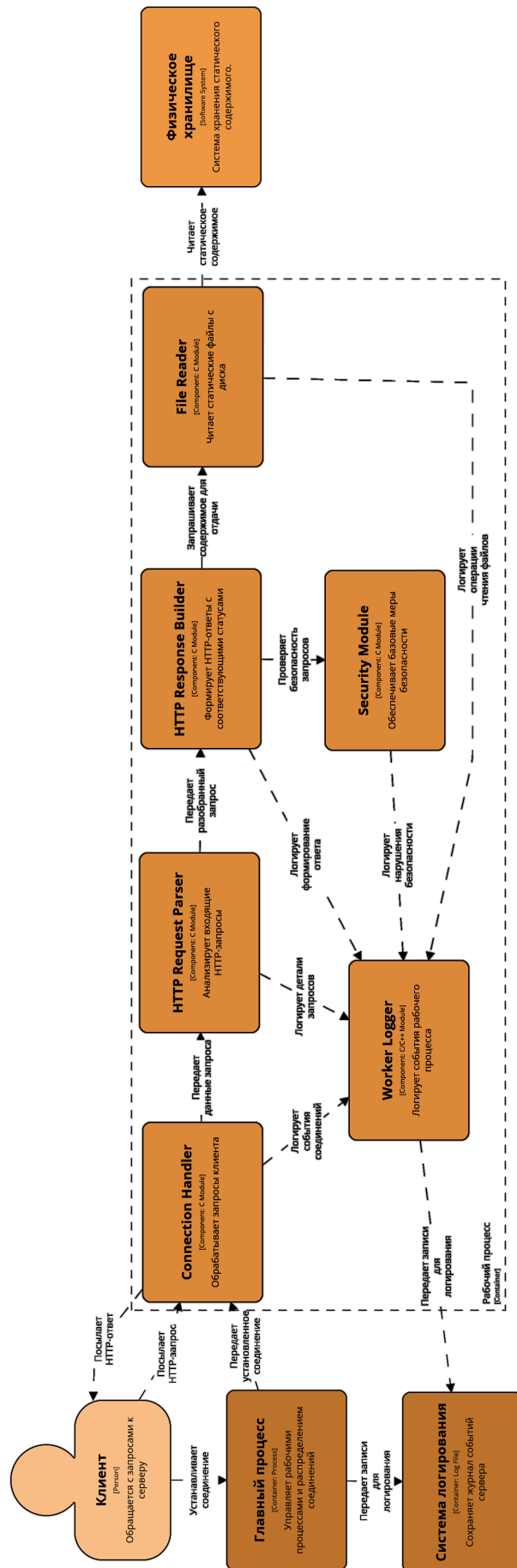


Рисунок 2.4 – Диаграмма компонентов рабочего процесса

2.3 Алгоритмы работы сервера

2.3.1 Алгоритм запуска сервера

Схема алгоритма запуска сервера представлена на рисунке 2.5.

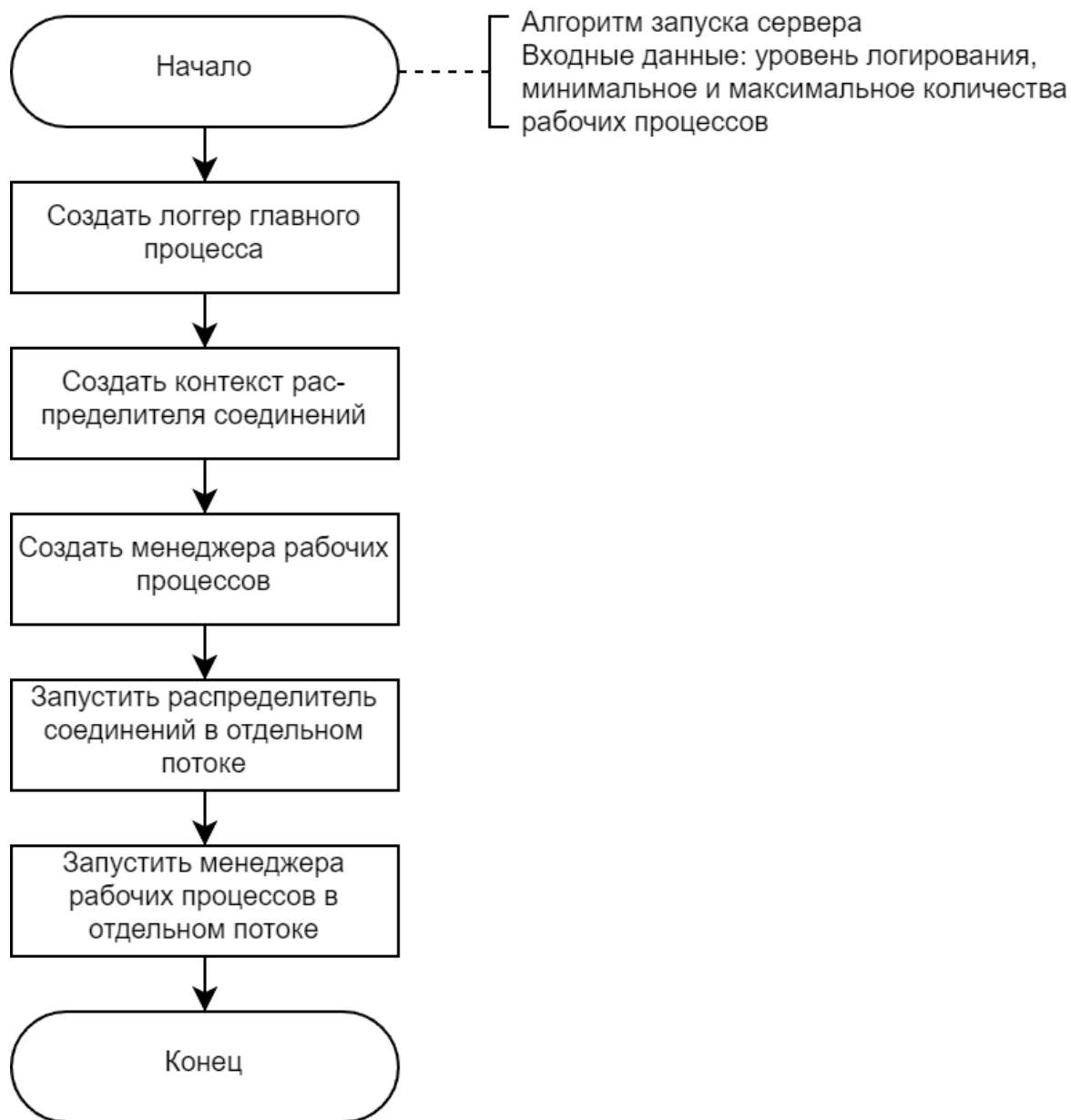


Рисунок 2.5 – Схема алгоритма запуска сервера

2.3.2 Алгоритм передачи нового соединения от главного процесса к рабочему

Схема алгоритма передачи нового соединения от главного процесса к рабочему представлена на рисунке 2.5.

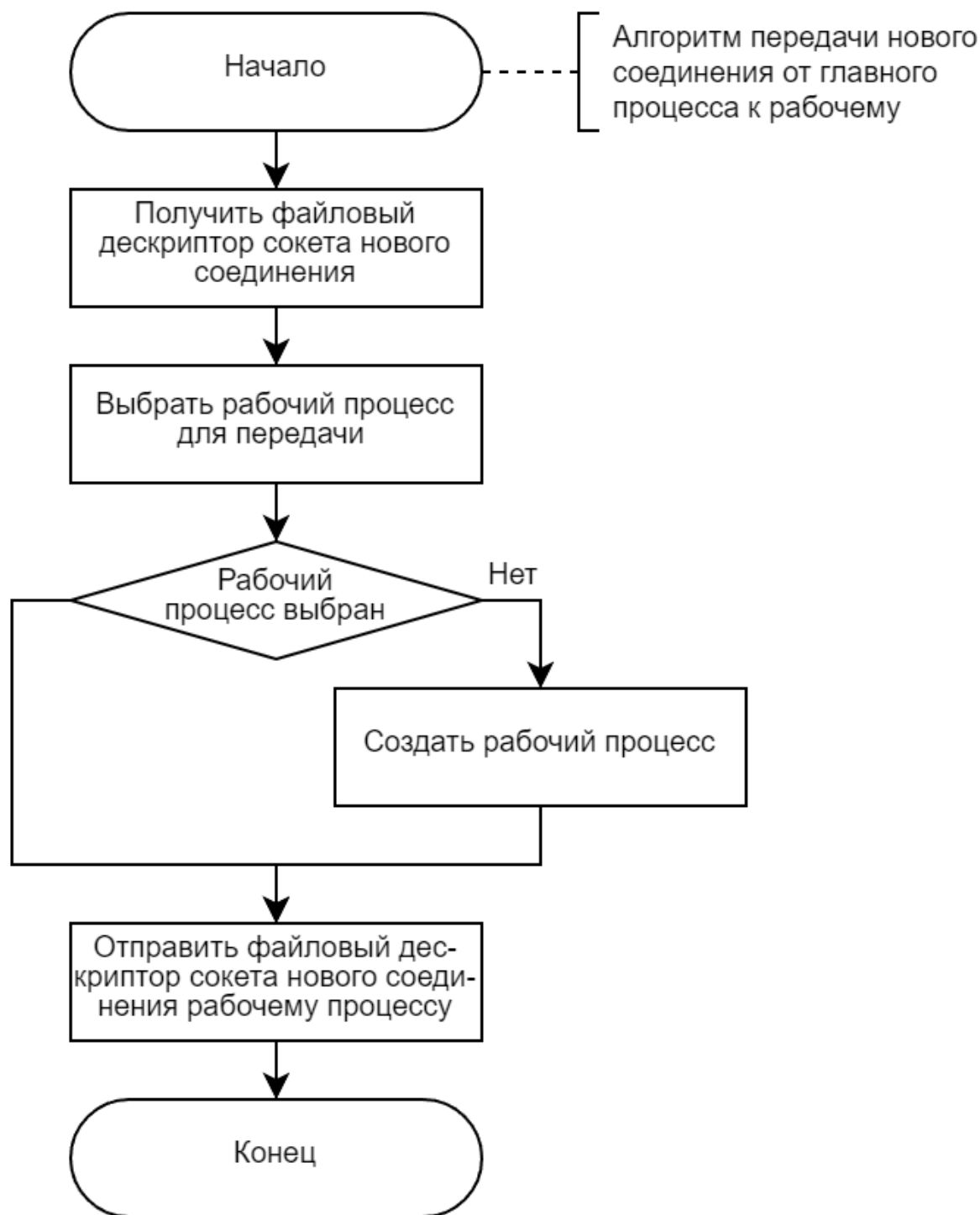


Рисунок 2.6 – Схема алгоритма передачи нового соединения от главного процесса к рабочему

2.3.3 Алгоритм обработки запроса к серверу

Схема алгоритма обработки запроса к серверу представлена на рисунке 2.5.

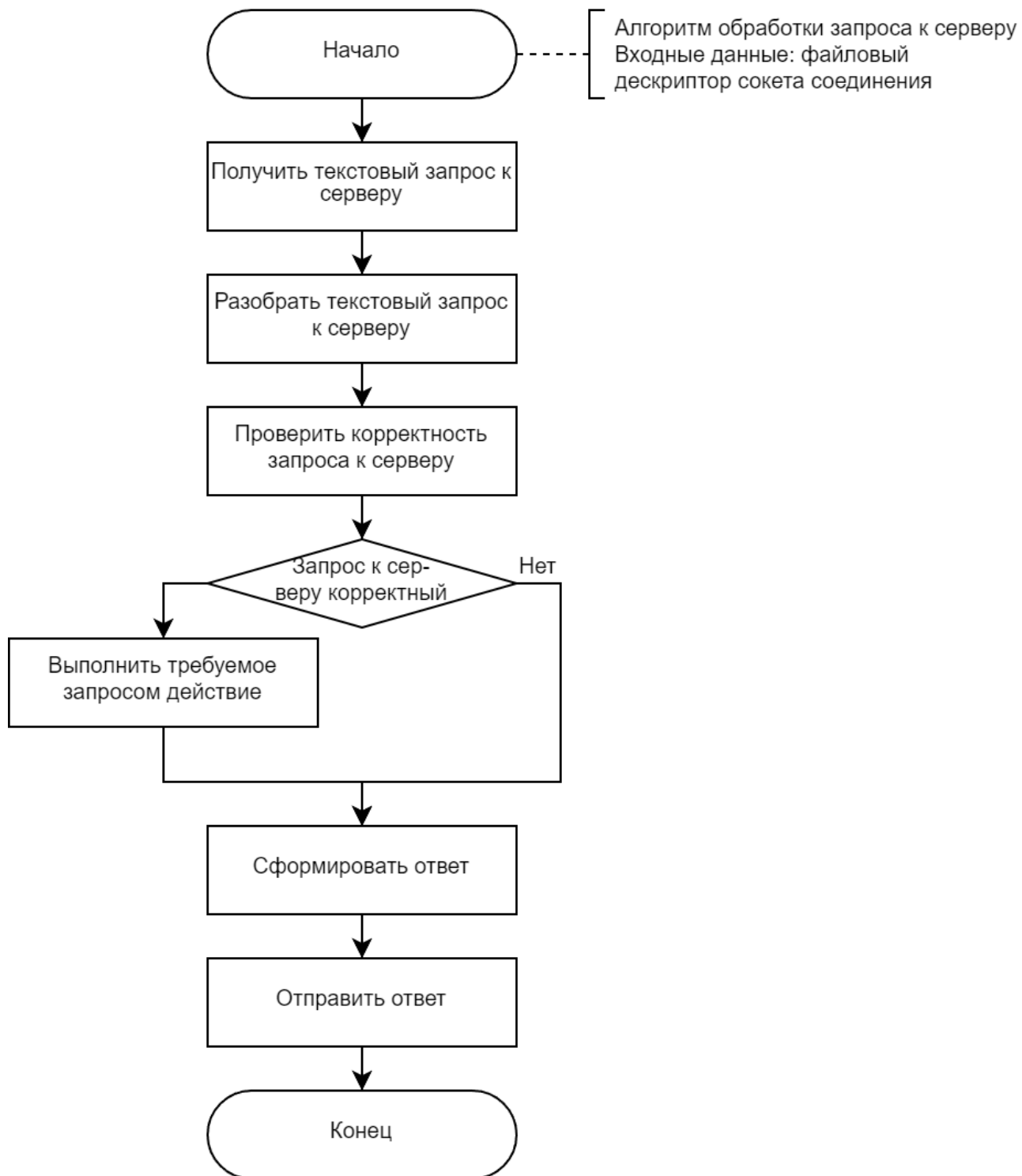


Рисунок 2.7 – Схема алгоритма обработки запроса к серверу

2.4 Выбор типов и структур данных

В набор выбранных структур данных включены:

1. Структура `distributor_cfg_t`, определяющая настройки распределителя соединений:
 - поле `listen_port` определяет порт для прослушивания;
 - поле `addr_info` определяет информацию об адресе.
2. Структура `distributor_ctxt_t`, определяющая контекст распределителя соединений:
 - поле `listen_sock` определяет сокет прослушивания;
 - поле `config` определяет настройки распределителя.
3. Структура `worker_info_t`, содержащая информацию о рабочем процессе:
 - поле `pid` определяет идентификатор процесса;
 - поле `to_worker_channel` определяет канал связи с процессом;
 - поле `logger` определяет систему ведения журнала событий.
4. Структура `worker_pool_t`, определяющая пул рабочих процессов:
 - поле `worker_count` определяет количество процессов;
 - поле `workers` определяет массив информации о процессах.
5. Структура `conn_ctxt_t` определяет контекст обработки подключений:
 - поле `read_fds` определяет набор файловых дескрипторов для чтения;
 - поле `max_fd` определяет максимальный номер дескриптора;
 - поле `conn_sockets` определяет массив файловых дескрипторов сокетов подключений;
 - поле `conn_sockets_count` определяет количество подключений.

6. Структура `http_request_t`, определяющая HTTP-запрос:

- поле `method` определяет метод запроса;
- поле `path` определяет путь запроса;
- поле `version` определяет версию протокола;
- поле `body` определяет тело запроса;
- поле `headers` определяет список заголовков.

7. Структура `http_response_t`, определяющая HTTP-ответ:

- поле `status_code` определяет код состояния;
- поле `headers` определяет список заголовков;
- поле `body` определяет тело ответа;
- поле `body_length` определяет длину тела.

2.5 Классы эквивалентности для тестирования

Классы эквивалентности для тестирования представлены в таблице 2.1.

Таблица 2.1 – Классы эквивалентности для тестирования сервера

Класс	Описание
Корректные HTTP-запросы	Корректные запросы с поддерживаемыми методами, правильными и допустимыми путями
Некорректные HTTP-запросы	Запросы с ошибками в синтаксисе, неподдерживаемыми методами
Штатная работа процессов	Стабильные создание, коммуникация и завершение рабочих процессов
Аварийные ситуации процессов	Аварийное завершение и исчерпание лимита процессов
Успешные HTTP-ответы	Ответы с кодом 200, корректными заголовками и содержимым
Ошибочные HTTP-ответы	Ответы с кодами ошибок, отсутствующими заголовками или некорректным содержимым
Непредельная нагрузка сервера	Стандартное количество подключений и относительно маленькая частота запросов
Предельная нагрузка сервера	Максимальное количество подключений и относительно высокая частота запросов

Вывод

В данном разделе были представлены требования к серверу, архитектура сервера, алгоритмы работы сервера, выбор типов и структур данных и классы эквивалентности для тестирования.

3 Технологический раздел

В данном разделе представляются средства реализации, демонстрация работы сервера и тестирование.

3.1 Средства реализации

В качестве целевой операционной системы для работы данного сервера была выбрана Linux Ubuntu [5] по следующим причинам:

- возможность реализовать механизм `prefork` благодаря наличию системного вызова `fork()` [6] в коде ядра;
- возможность реализовать механизм `select` благодаря наличию системного вызова `select()` [7] в коде ядра;
- возможность реализовать механизм межпроцессного взаимодействия [3].

В качестве языка программирования для реализации данного сервера был выбран C [8] по следующим причинам:

- в стандартной библиотеке языка присутствуют типы и структуры данных, выбранные по результатам проектирования;
- код ядра целевой операционной системы написан на данном языке.

Исходный код разработанного сервера доступен в открытом сетевом репозитории [9].

3.2 Демонстрация работы сервера

Пример работы сервера при запуске представлен в листинге 3.1.

Листинг 3.1 – Пример работы сервера при запуске

```
1 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969] main: logger
   created
2 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   distributor_socket_create: socket created successfully on
   port 8080
3 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   distributor_ctxt_create: connection distributor created
   successfully
```

```

4 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969] main:
   connection distributor created
5 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_processes_create: creating 3 worker processes
6 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_process_create: created worker process with PID
   50970
7 [2025-10-01 00:57:30] [DEBUG] [MASTER] [PID:50969]
   worker_pool_processes_create: worker 0 created with PID 50970
8 [2025-10-01 00:57:30] [INFO] [WORKER] [PID:50970]
   connection_handler_handler: worker process with PID 50970
   started
9 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_process_create: created worker process with PID
   50971
10 [2025-10-01 00:57:30] [DEBUG] [MASTER] [PID:50969]
   worker_pool_processes_create: worker 1 created with PID 50971
11 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_process_create: created worker process with PID
   50972
12 [2025-10-01 00:57:30] [INFO] [WORKER] [PID:50971]
   connection_handler_handler: worker process with PID 50971
   started
13 [2025-10-01 00:57:30] [INFO] [WORKER] [PID:50972]
   connection_handler_handler: worker process with PID 50972
   started
14 [2025-10-01 00:57:30] [DEBUG] [MASTER] [PID:50969]
   worker_pool_processes_create: worker 2 created with PID 50972
15 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_processes_create: successfully created 3 worker
   processes
16 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969] main: worker
   pool created
17 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969] main:
   distributor thread created
18 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969] main: worker
   pool thread created
19 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   distributor_listening: starting connection distribution
20 [2025-10-01 00:57:30] [INFO] [MASTER] [PID:50969]
   worker_pool_monitoring: starting worker pool monitoring

```

Пример отправки HEAD-запроса на сервер представлен в листинге 3.2.

Листинг 3.2 – Пример отправки HEAD-запроса на сервер

```
1 | kori@kori:~/bmstu/cw_cn/test$ curl
   http://localhost:8080/test.txt -I
2 | HTTP/1.1 200 OK
3 | Server: Simple HTTP Server
4 | Connection: close
5 | Content-Type: text/plain
6 | Content-Length: 11
```

Пример отправки GET-запроса на сервер представлен в листинге 3.3.

Листинг 3.3 – Пример отправки GET-запроса на сервер

```
1 | kori@kori:~/bmstu/cw_cn/test$ curl
   http://localhost:8080/test.txt -O
2 | % Total % Received % Xferd Average Speed Time Time Time Current
3 |                                Dload Upload Total Spent Left Speed
4 | 100 11 100 11 0 0 779 0 --:--:-- --:--:-- --:--:-- 846
```

Пример отправки POST-запроса на сервер представлен в листинге 3.4.

Листинг 3.4 – Пример отправки POST-запроса на сервер

```
1 | kori@kori:~/bmstu/cw_cn/test$ curl -X POST -d {"data": "value"}
   http://localhost:8080/
2 | HTTP/1.1 405 Method Not Allowed
3 | Server: Simple HTTP Server
4 | Connection: close
5 | Allowed: GET, HEAD
```

Пример работы сервера при обработки HEAD-запроса представлен в листинге 3.5.

Листинг 3.5 – Пример работы сервера при обработки HEAD-запроса

```
1 | [2025-10-01 01:01:05] [INFO] [MASTER] [PID:50969]
   distributor_accept_connection: accepted connection from
   127.0.0.1:35566 (socket_fd 10)
2 | [2025-10-01 01:01:05] [INFO] [MASTER] [PID:50969]
   distributor_conn_delegate: delegated connection to worker
   with PID 50972
3 | [2025-10-01 01:01:05] [INFO] [WORKER] [PID:50972]
   connection_handler_new_connection_handle: new client
   connected, socket_fd 10
4 | [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
   connection_handler_recieve_request: request text - HEAD
```



```

    /test.txt HTTP/1.1
5 Host: localhost:8080
6 User-Agent: curl/7.81.0
7 Accept: */*
8
9
10 [2025-10-01 01:01:05] [INFO] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request from sock_fd
    10 succesfully recieved
11 [2025-10-01 01:01:05] [INFO] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request from sock_fd
    10 succesfully parsed
12 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request method:
    "HEAD"
13 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request path:
    "/test.txt"
14 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request version:
    "HTTP/1.1"
15 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request header:
    "Host: localhost:8080"
16 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request header:
    "User-Agent: curl/7.81.0"
17 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_http_request_handle: request header:
    "Accept: */*"
18 [2025-10-01 01:01:05] [INFO] [WORKER] [PID:50972]
    connection_handler_http_request_handle: response for sock_fd
    10 succesfully created
19 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_send_response: http_response_text -
    HTTP/1.1 200 OK
20 Server: Simple HTTP Server
21 Connection: close
22 Content-Type: text/plain
23 Content-Length: 11
24

```

```

25
26 [2025-10-01 01:01:05] [DEBUG] [WORKER] [PID:50972]
    connection_handler_send_response: body length - 0
27 [2025-10-01 01:01:05] [INFO] [WORKER] [PID:50972]
    connection_handler_http_request_handle: response for sock_fd
    10 succesfully has been sent

```

3.3 Тестирование

Результаты тестирования работы сервера представлены в таблице 3.1.

Таблица 3.1 – Результаты тестирования работы сервера

Тест	Ожидаемый результат	Фактический результат
GET запрос к существующему файлу	HTTP 200 с содержимым файла	HTTP 200 с содержимым файла
GET запрос к несуществующему файлу	HTTP 404 Not Found	HTTP 404 Not Found
Запрос с неподдерживаемым методом	HTTP 405 Method Not Allowed	HTTP 405 Method Not Allowed
Path traversal атака	HTTP 403 Forbidden	HTTP 403 Forbidden
GET запрос к директории	HTTP 200 с содержимым директории	HTTP 200 с содержимым директории
URI превышающий максимальную длину	HTTP 414 URI Too Long	HTTP 414 URI Too Long
Запрос файла больше максимального размера	HTTP 413 Payload Too Large	HTTP 413 Payload Too Large
Множественные одновременные подключения	Корректная обработка всех запросов	Корректная обработка всех запросов
Запрос к файлу с разными MIME-типами	Правильное определение Content-Type	Правильное определение Content-Type
Запрос при максимальной нагрузке	Стабильная работа без сбоев	Стабильная работа без сбоев
Логирование различных событий	Запись сообщений в указанную цель	Запись сообщений в указанную цель

Все тесты были успешно пройдены.

Вывод

В данном разделе были представлены средства реализации, демонстрация работы сервера и тестирование.

4 Исследовательский раздел

В данном разделе представляются постановка и проведение исследования работы сервера.

4.1 Постановка исследования

Целью исследования работы сервера является определение следующих значений:

- максимальное количество обслуживаемых сервером сетевых соединений;
- скорость отдачи данных по сетевому соединению;
- совокупная скорость отдачи данных по всем соединениям.

4.2 Технические характеристики

Спецификации устройства, использованного для проведения исследования:

- оперативная память 6 ГБ;
- процессор Intel(R) Core(TM) i7-9750H с тактовой частотой 2.60ГГц;
- 3 физических и 5 логических ядер;
- операционная система Linux Ubuntu 22.04.1 LTS 64-разрядная с версией ядра 6.8.0-84-generic.

Устройство было нагружено только встроенными приложениями и подключено в сеть электропитания.

4.3 Проведение исследования

Для проведения исследования работы сервера используется однопоточная программа для командной строки Apache Benchmark [10].

4.3.1 Максимальное количество обслуживаемых сетевых соединений

Теоретически максимальное количество обслуживаемых сервером сетевых соединений определяется несколькими характеристиками:

- максимальным количеством открытых процессом файловых дескрипторов;
- параметром сервера `MAX_CONNECTIONS`, определяющим максимальное количество обслуживаемых сетевых соединений рабочим процессом;
- параметром сервера `MAX_WORKERS`, определяющим максимальное число рабочих процессов.

Теоретическое значение максимального количества обслуживаемых сетевых соединений $N_{server_max_connections}$ может быть вычислено следующим образом:

$$N_{server_max_connections} = N_{max_workers} \cdot \min(N_{max_connections}, N_{open_files}) \quad (4.1)$$

где:

1. $N_{max_workers}$ – максимальное число рабочих процессов;
2. $N_{max_connections}$ – максимальное количество обслуживаемых сетевых соединений рабочим процессом;
3. N_{open_files} – максимальное количество открытых процессом файловых дескрипторов.

Значения характеристик исследуемого сервера приведены в таблице 4.1.

Таблица 4.1 – Значения характеристик для определения максимального количества обслуживаемых сервером сетевых соединений

Характеристика	Значение, шт.
Максимальное количество открытых процессом файловых дескрипторов	1024
<code>MAX_CONNECTIONS</code>	1024
<code>MAX_WORKERS</code>	10

Рассчитанное теоретическое значение максимального количества обслуживаемых сетевых соединений составляет **10240**.

Практически максимальное количество соединений определяется с помощью нагрузочного тестирования. Количество подключений к серверу увеличивается до тех пор, пока сервер не перестанет справляться с нагрузкой. Критерием достижения максимума является появление ошибок.

Практически полученное значение максимального количества обслуживаемых сетевых соединений составляет **10240**.

Практическое значение максимального количества обслуживаемых сетевых соединений соответствует теоретическому, поскольку теоретическое значение максимального количества обслуживаемых сетевых соединений явно передаётся в качестве параметра системному вызову `listen()`. Передача данного параметра представлена в листинге 4.1.

Листинг 4.1 – Использование системного вызова `listen()`

```
1 // connection_distributor.c
2 if (IS_ERROR(listen(listen_sock, MAX_WORKERS *
3     MAX_CONNECTIONS))) {
4     LOG_ERROR(logger, "distributor_socket_create error: failed
5         to listen on socket: %s", strerror(errno));
6     close(listen_sock);
7     return EXIT_FAILURE;
8 }
```

4.3.2 Скорость отдачи данных по сетевому соединению

Скорость отдачи данных по сетевому соединению определяется с помощью нагрузочного тестирования. Клиент посылает GET-запросы к серверу, целью запроса является файл объемом 680 КБ, количество запросов к серверу увеличивается с шагом 100.

Результаты определения скорости отдачи данных по сетевому соединению представлены в таблице 4.2.

По таблице 4.2 был построен график 4.1 зависимости времени отдачи данных от количества запросов. Исходя из графика, можно сделать вывод, что данная зависимость близка к линейной.

Скорость отдачи данных по сетевому соединению находится в интервале от 155 Мбайт/сек до 204 Мбайт/сек, среднее значение составляет **180 Мбайт/сек**.

Таблица 4.2 – Результаты определения скорости отдачи данных по сетевому соединению

Количество запросов, шт.	Скорость отдачи данных, Кбайт/сек	Время отдачи данных, с
100	183405	0.363
200	199842	0.771
300	179200	1.113
400	159101	1.643
500	192682	1.925
600	177470	2.248
700	172546	2.667
800	182010	2.922
900	189677	3.155
1000	209451	3.374

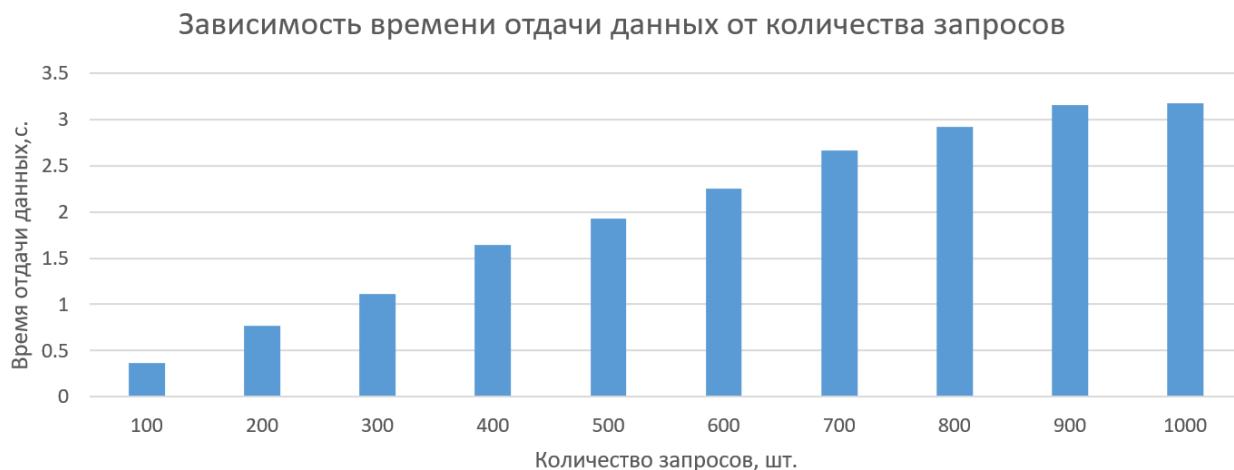


Рисунок 4.1 – График зависимости времени отдачи данных от количества запросов

4.3.3 Совокупная скорость отдачи данных по всем сетевым соединениям

Скорость отдачи данных по всем сетевым соединениям определяется с помощью нагрузочного тестирования. Клиенты посылают 1000 GET-запросов к серверу, целью запроса является файл объемом 680 КБ, количество клиентов увеличивается с шагом 100.

Результаты определения скорости отдачи данных по всем сетевым соединениям представлены в таблице 4.3.

По таблице 4.3 был построен график 4.2 зависимости времени отдачи данных от количества клиентов. Исходя из графика, можно сделать вывод, что данная зависимость близка к линейной.

Таблица 4.3 – Результаты определения скорости отдачи данных по всем сетевым соединениям

Количество клиентов, шт	Совокупная скорость отдачи данных, Кбайт/сек	Время отдачи данных, с
100	204572	3.324
200	203471	3.342
300	192036	3.541
400	189891	3.581
500	184883	3.678
600	181867	3.739
700	179704	3.784
800	176715	3.848
900	174136	3.905
1000	172633	3.939



Рисунок 4.2 – График зависимости времени отдачи данных от количества клиентов

Вывод

В данном разделе были представлены постановка и проведение исследования работы сервера. В ходе исследования были определены значение максимальное количество обслуживаемых сервером сетевых соединений, зависимости времени отдачи данных от количества запросов и времени отдачи данных от количества клиентов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы был разработан сервер для отдачи статического содержимого с диска по протоколу HTTP с использованием механизмов `prefork` и `select`.

Для достижения поставленной цели необходимо были выполнены следующие задачи:

- проведен анализ предметной области;
- проведен анализ используемого набора механизмов;
- спроектирована архитектура разрабатываемого сервера;
- выбраны средства реализации и реализован сервер;
- проведено тестирование разработанного сервера;
- проведено исследование разработанного сервера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Тихомирова Е.А.* Конспект лекций по курсу «Компьютерные сети». — МГТУ им. Н.Э. Баумана, 2025.
2. *Бекасов Д.Е.* Конспект лекций по курсу «Основы разработки Web-приложений». — МГТУ им. Н.Э. Баумана, 2025.
3. *Рязанова Н.Ю.* Конспект лекций по курсу «Операционные системы». — МГТУ им. Н.Э. Баумана, 2025.
4. Документация по нотации моделирования архитектуры программных систем C4. — URL: <https://c4model.com/> (дата обращения: 15.09.2025).
5. Документация по операционной системе Linux. — URL: <https://help.ubuntu.com/> (дата обращения: 01.10.2025).
6. Linux manual page about fork(). — URL: <https://man7.org/linux/man-pages/man2/fork.2.html/> (дата обращения: 01.10.2025).
7. Linux manual page about select(). — URL: https://man7.org/linux/man-pages/man2/select.2.html (дата обращения: 01.10.2025).
8. GNU C Documentation. — URL: <https://www.gnu.org/software/gnu-c-manual/gnu-c-manual.html> (дата обращения: 01.10.2025).
9. Исходный код сервера. — URL: https://github.com/Kori-Tamashi/bmstu/tree/course_work_cn/course_works/computer_networks/code (дата обращения: 01.10.2025).
10. Apache Benchmark Documentation. — URL: <https://httpd.apache.org/docs/2.4/programs/ab.html> (дата обращения: 01.10.2025).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе, содержащая n слайдов